Journal of Systems Architecture 170 (2026) 103630

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

LogHGAD: A hypergraph-based log anomaly detection method for AIoT systems☆

Jiewei He, Chao Song *, Ruilin Hu, Zheng Ren

School of Computer Science and Technology, University of Electronic Science and Technology of China, Chengdu, 611731, China

ARTICLE INFO

Keywords:
AIoT security
Context information integration
Hypergraph
Hypergraph convolutional network
Log anomaly detection
Multivariate relationships

ABSTRACT

In AIoT-enabled systems, log-based anomaly detection is critical for ensuring system security and reliability. Existing methods struggle with the concurrent, interleaved nature of logs from modern software systems and often neglect context information in log headers and parameters. To address these limitations, we propose LogHGAD, a novel log anomaly detection method based on hypergraph modeling. LogHGAD partitions logs into session windows using identifiers and constructs a hypergraph where nodes represent log events (with semantic embeddings as features) and hyperedges capture both temporal sequences and complex multivariate interactions (e.g., between processes, components, and entities). It uniquely integrates log header information (e.g., PID, component) and log parameters into the model. An end-to-end framework using a hypergraph convolutional network learns structural embeddings, fuses them with semantic features, and employs downstream classifiers for detection. Extensive experiments on five real-world log datasets demonstrate that LogHGAD outperforms seven state-of-the-art baselines in accuracy, precision, recall, specificity, and F1-Score, while maintaining practical efficiency.

1. Introduction

In the AIoT-enabled smart society [1–5], systems and devices continuously generate massive logs during operation. These logs record data interactions, device state changes, and system event trajectories in detail, forming the core data source for post-hoc analysis. As the last critical line of defense for system security, log-based analysis [6–10] not only helps trace equipment failures and optimize service processes but also plays a decisive role in security scenarios such as cyber attack tracing and privacy breach detection. IoT devices collect sensitive data, such as biometric information and geographic location. These data, along with the processing flows of AI algorithms, are digitally recorded in logs. Consequently, any abnormal patterns within these logs may reveal potential security vulnerabilities or privacy violations. Therefore, strengthening the research on log anomaly detection technologies and improving the timeliness and accuracy of log analysis have become necessary preconditions for building a solid security barrier for AIoT systems and achieving the sustainable development of smart societies [11–13].

Logs are semi-structured data dynamically generated during system operation [14]. They meticulously record various information such as user operations, system state changes, and error warnings, providing key clues for understanding internal system mechanisms. However, diagnosing system status through log analysis still faces significant challenges [14–17]. Application systems are often developed by multiple system engineers with varying levels of experience. This leads to logs of inconsistent quality being generated during system operation. Meanwhile, due to the rapid increase in system log scale (e.g., 50GB/hour), even experienced engineers face huge efficiency bottlenecks when analyzing and handling anomalies through logs. Therefore, in recent years, an increasing number of studies have focused on the field of automated log analysis [18–21], and log-based anomaly detection is a key task among them [22,23].

Current research on log-based anomaly detection has evolved from early machine learning methods [24–26] to deep learning-based sequence modeling methods [27–31], which have demonstrated progress in capturing log semantics and temporal dependencies. Although the aforementioned methods have achieved promising results to a certain extent, they still suffer from the following issues: (1) **Concurrency and interactivity challenges.** Modern systems execute concurrently, resulting in diverse log message sources and operation objects, with log messages appearing interleaved. Session-window based methods partition log datasets using predefined identifiers. However, they focus only on a single message source or operation object. Given the complex relationships within systems, a single identifier cannot adequately describe

☆ This article is part of a Special issue entitled: 'SPASS' published in Journal of Systems Architecture.

* Corresponding author.
E-mail address: chaosong@uestc.edu.cn (C. Song).

https://doi.org/10.1016/j.sysarc.2025.103630
Received 30 June 2025; Received in revised form 9 November 2025; Accepted 15 November 2025
Available online 16 November 2025
1383-7621/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

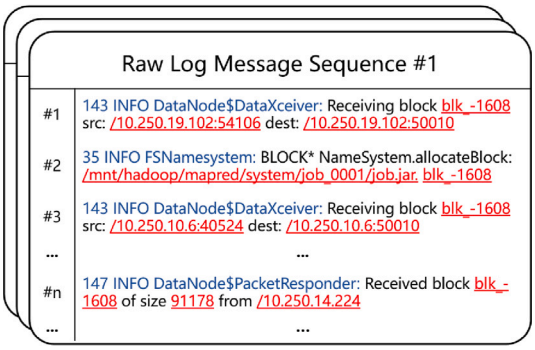


Fig. 1. The partial raw log message sequence in HDFS dataset.

the internal multivariate relationships, while some approaches even omit partitioning altogether, directly compromising the authenticity and validity of model results. *These limitations reduce model accuracy and weaken anomaly detection reliability*. (2) **Neglect of context information.** Existing methods primarily analyze log events while neglecting valuable context information in log headers (e.g., log levels, process IDs, and components). For instance, in Fig. 1, HDFS log headers in #1 reveal system status ("INFO"), process details (PID 143), and active components ("DataNode$DataXceiver")—critical for anomaly detection. As the system runs, this header context information evolves. *Ignoring log headers means missing critical anomaly detection clues, since headers contain complex multivariate relationships within the system.*

In this paper, we propose a novel hypergraph-based log anomaly detection method named LogHGAD (Log Hypergraph Anomaly Detection) to address the above issues. We partition log messages into session windows using specific identifiers, construct a hypergraph for each window's log sequence, where nodes represent log events with semantic embeddings and hyperedges capture temporal and multivariate relationships. Our method integrates log header information and parameters overlooked by prior work, enabling a more comprehensive representation of system operations. Leveraging hypergraph convolutional networks, we develop an end-to-end anomaly detection model. Experiments on multiple datasets show that LogHGAD outperforms state-of-the-art methods.

The main contributions of this paper are as follows:

- **Hypergraph modeling for multivariate relationships.** We introduce hypergraph structures to model log sessions, effectively capturing the complex multivariate relationships (e.g., interactions among heterogeneous components and objects) in concurrent systems, overcoming traditional single-identifier limitations for more precise system representation.
- **Integration of log context information.** We incorporate log headers (e.g., process IDs, components) and parameters alongside log events, enriching system state representation and improving anomaly detection with previously ignored contextual cues.
- **Efficient end-to-end anomaly detection model.** We propose an end-to-end anomaly detection model based on hypergraph convolutional network, outperforming seven baseline methods across five real-world datasets while maintaining computational efficiency. Ablation studies further validate the effectiveness of our log hypergraph model.

The rest of the paper is organized as follows. Section 2 introduces the related work. Section 3 presents background and motivation of this work. Section 4 describes the details of LogHGAD. Section 5 represents the experiments and corresponding results.

2. Related work

Log-based anomaly detection research has evolved through three primary stages [32]: machine learning, deep learning, and more recently, graph-based methods.

Machine learning methods. Early methods based on machine learning achieve anomaly detection by extracting quantitative features of log events within specific log window sequences, such as event count, occurrence frequency, and numerical parameter distributions. Xu et al. [24] introduced a PCA-based technique that models normal log patterns via dimensionality reduction, while Lin et al. [25] proposed LogCluster, which uses clustering on event vectors to identify anomalies. Lou et al. [26] mined linear invariants from logs to detect deviations. Nevertheless, these methods only focus on the statistical patterns of log sequence data, ignoring the sequential relationships between log records and the inherent semantic information of log text, resulting in limited adaptability and accuracy.

Deep learning methods. To capture richer semantic and sequential patterns, some studies [27–31,33,34] have applied deep learning technologies to log anomaly detection with promising results. Most of these methods treat logs as natural language sequences [27–31], incorporating the semantics of log events through sequence models to mine the logical order and contextual dependencies of log events for determining the current system operation. DeepLog [27] frames log anomaly detection as an LSTM multi-class prediction task, flagging anomalies when real events deviate from the forecasted top candidates. LogAnomaly [28] combines semantic and quantitative LSTMs for dual-mode anomaly detection, while LogRobust [29] employs attention-enhanced Bi-LSTM to process log semantic sequences. Both methods integrate semantic and statistical features for robust anomaly detection. PLELog [30] introduces a semi-supervised approach using HDBSCAN clustering and GRU with attention to learn from minimally labeled data. However, these methods primarily focus on the sequential patterns of logs while ignoring the complexity of system processes, prone to overfitting. Moreover, these models often demand substantial computational resources.

Graph-based methods. Recently, researchers have begun to explore the application of graph models in log anomaly detection. Zhang et al. [35] built a log graph based on invocation relationships among microservices, and introduced DeepTraLog, a method that employs a Gated Graph Sequence Neural Network and Deep Support Vector Data Description (Deep SVDD) for log anomaly detection. Xie et al. [36] proposed LogGD, transforming logs into weighted directed graphs and integrating structural and semantic features via graph Transformers, achieving anomaly detection via graph classification. Li et al. [37] introduced Logs2Graphs, which converts log sequences into attributed directed graphs, uses a Digraph Inception Convolutional Network for multi-scale feature extraction, and applies Deep SVDD in the graph representation space. These methods represent valuable explorations in applying graph-based techniques to log anomaly detection. They treat log events as nodes and the edges between nodes as event transitions. However, in essence, they remain an enhancement of sequential methods.

While existing methods have made significant progress in log anomaly detection, they primarily focus on either statistical relationships or sequential patterns, often overlooking the complex multivariate interactions inherent in modern distributed systems. In contrast, LogHGAD introduces a hypergraph-based framework that uniquely addresses three key limitations: (1) **Multivariate relationship modeling.** Unlike traditional methods relying on single identifiers or linear sequences, LogHGAD captures concurrent interactions among processes, components, and entities through hyperedges, enabling a more accurate representation of system dynamics. (2) **Information integration.** Existing studies largely overlook log headers (e.g., process IDs, component names) and parameters, whereas LogHGAD explicitly integrates these features to enhance detection granularity. (3) **Accuracy-efficiency**

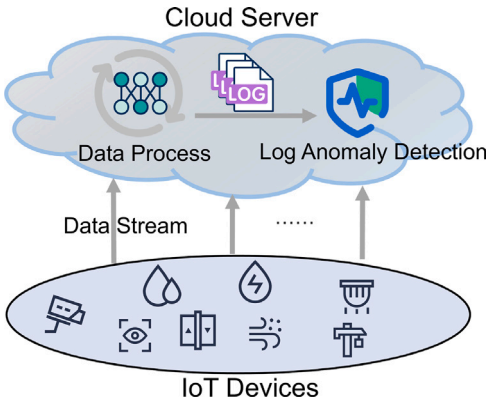


Fig. 2. An example of log processing in IoT systems.

balance. Addressing the high computational costs and suboptimal accuracy of deep sequence models, LogHGAD employs hypergraph convolutional networks to maintain computational efficiency while achieving superior accuracy. Our work bridges the gap between fine-grained system behavior modeling and practical deployability, outperforming existing methods across diverse datasets.

3. Background and motivation

In this section, we first introduce the log terminology, the process of log parsing and structuring, as well as the hypergraph, and then further elaborate on the motivation behind this research. Moreover, we define several important terminologies pertinent to our work. Frequently used notations in this paper are summarized in Table 1.

3.1. Log terminology

In large-scale IoT systems, as shown in Fig. 2, massive volumes of data generated by numerous IoT devices are transmitted to cloud servers for processing and log generation. The raw log message sequence consists of line-by-line log entries, recording rich runtime state information of the system. Each log message is a semi-structured text record, including a structured log header (blue text in Fig. 1) and a semi-structured log body (text following the blue text in Fig. 1). The log header generally contains information such as log level, process ID, and the current running system component, with the specific structure determined by the logging tool used by the system. The log body, defined by software engineers during programming, can be divided into a constant part and a variable part.

The constant part of the log body is typically referred to as the log event or log template [29,31]. For example, the log body of log #1 in Fig. 1, "Receiving block blk_-1608 src: /10.250.19.102:54106 dest: /10.250.19.102:50010", is parsed to extract the log event "Receiving block <*> src: <*> dest: <*>". The variable part of the log body, namely log parameters, is automatically generated during system operation to fill the placeholders in the log template. Logs with the same event template may contain varying parameters, reflecting real-time system states. If an anomaly-level log message exists in the log sequence, the sequence is labeled as an anomalous sequence; otherwise, it is a normal sequence [30].

3.2. Log parsing and structuring

As shown in Fig. 3, the log parsing and structuring process transforms the raw semi-structured log sequence into a structured tabular data structure (log dataframe), which supports subsequent log representation and anomaly detection tasks. Since the original source code of the application generating the log dataset is often unavailable, existing studies typically employ log parsing tools to extract log events, with the extracted events approximating those defined during programming. We use Drain [20] as the parsing tool, as it has demonstrated superior performance [38–40] and is widely adopted in numerous log anomaly detection studies [22,23].

Table 1
Notation description.

Symbol	Description
$\mathcal{G}$	Hypergraph structure
$\mathcal{V}$	Set of nodes in the hypergraph
$\mathcal{E}$	Set of hyperedges in the hypergraph
$\mathcal{W}$	Diagonal weight matrix for hyperedges
$\mathcal{H}$	Incidence matrix representing node-hyperedge associations
$h(v,e)$	Entries of $\mathcal{H}$ indicating whether the node v is connected by the hyperedge e
$\mathcal{D}$	Node degree matrix
$\mathcal{B}$	Hyperedge degree matrix
$\mathcal{X}^l$	Node feature matrix at layer l
θ	Weight matrix between network layers
σ	Non-linear activation function
$\mathcal{Z}$	Final representation of the hypergraph
c	Center of the hypersphere in Deep SVDD
$\mathcal{R}$	Radius of the hypersphere in Deep SVDD
τ	Boundary balancing parameter in Deep SVDD loss
λ	Weight decay parameter for regularization
$L_i \in L$	The ith log message in the log window sequence
v_i	Log node representing log message L_i
se_i	Sequential hyperedge connecting v_{i-1} and v_i
$r \in \mathbb{R}$	The ith interaction relation in the set of interaction relations
ie_r	Interactive hyperedge for relation r
α	Positive/negative sample weight balance factor in Focal Loss
γ	Modulation factor for easy/hard samples in Focal Loss
TP	True positives in evaluation metrics
FP	False positives in evaluation metrics
TN	True negatives in evaluation metrics
FN	False negatives in evaluation metrics

3.3. Hypergraph

Hypergraph [41–44] is a generalized graph structure in graph theory. Unlike ordinary graphs where each edge connects exactly two nodes, hypergraphs feature hyperedges that can connect any number of nodes simultaneously. This flexibility enables hypergraphs to naturally describe complex relationships, making them suitable for modeling in multiple domains. A hypergraph can be defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{W}, \mathcal{H})$, where $\mathcal{V}$ is the set of nodes, $\mathcal{E}$ is the set of hyperedges, $\mathcal{W}$ is a diagonal weight matrix assigning a positive weight $\mathcal{W}_{ee}$ to each hyperedge e, H is an incidence matrix of size $|\mathcal{V}| \times |\mathcal{E}|$ representing node-hyperedge associations, whose entries are defined as

$$h(v,e) = \begin{cases} 1 & \text{if } v \in e \\ 0 & \text{if } v \notin e \end{cases} \tag{1}$$

For $\forall v \in \mathcal{V}$, its degree is given by $\mathcal{D}_v = \sum_{e \in \mathcal{E}} \mathcal{W}_{ee} h(v,e)$. For $\forall e \in \mathcal{E}$, its degree is given by $\mathcal{B}_e = \sum_{v \in \mathcal{V}} h(v,e)$. The node degree matrix $\mathcal{D}$ and hyperedge degree matrix $\mathcal{B}$ are both diagonal matrices.

3.4. Motivation

To further elaborate on the concurrent and interleaved phenomena of log messages output and collected by software systems, we first define **log context switching** as follows:

Definition 1 (*Log Context Switching*)**.** For a specified log message, if its message source or operation object (extracted from the log header) differs from the previous adjacent timestamped log message, the context of the current log message is considered switched.

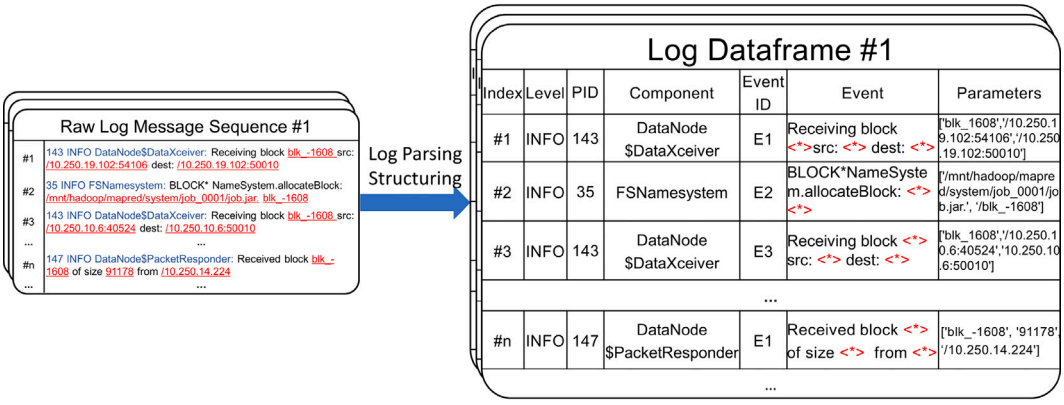


Fig. 3. An illustrating example for log parsing and structuring.

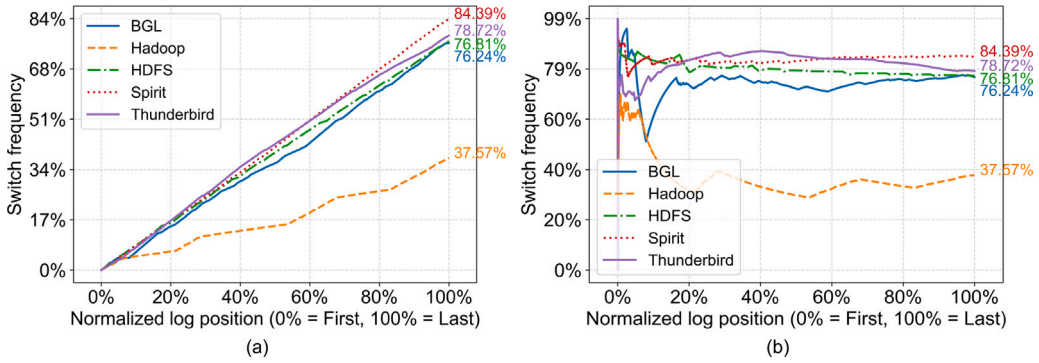


Fig. 4. Change diagrams of log context switching frequency for five log datasets. (a) Normalized cumulative change diagram. (b) Instantaneous change diagram.

Based on Definition 1, we conducted distribution statistics on the log datasets used in experimental evaluations, as shown in Fig. 4. The abscissa of both subplots in Fig. 4 represents the normalized log position, where each point indicates the relative position of the current log sequence number from the first log. Fig. 4(a) shows the cumulative change curve of log context switching frequency, where each point on the curve represents the percentage of log messages with context switches from the first log to the current position. It can be seen that except for the Hadoop dataset, where context-switched logs account for only 37.57%, the switching frequencies of the other four datasets are approximately 80%. Fig. 4(b) is an instantaneous change trend diagram of log context switching frequency, supplementing Fig. 4(a) to demonstrate sudden changes in switching behavior. Each point on the curve represents the proportion of cumulative context switches to the total number of logs in the current stream: an upward trend indicates increased recent switching frequency, while a downward trend indicates a decrease. As shown in Fig. 4(b), the curve of each dataset rises steeply in the early stage, fluctuates in the later stage, and finally stabilizes—indicating that context switches occur frequently at the beginning of the log stream, and the overall switching frequency gradually stabilizes as the number of logs increases.

The widespread phenomenon of frequent log context switching, as evidenced in Fig. 4, presents a fundamental challenge to traditional log anomaly detection methods based on sequential models. These methods inherently assume that adjacent log messages are semantically consecutive and relevant. However, a high context-switching rate directly violates this assumption, as adjacent logs likely originate from different processes, components, or operational contexts. This interleaving of disparate execution flows severely disrupts the learning of coherent temporal patterns, leading to degraded model performance. It is precisely this prevalence of concurrent and interleaved interactions that motivates our hypergraph design, particularly the introduction of interactive hyperedges. Unlike edges in ordinary graphs or sequences in RNNs, a hyperedge can connect any number of non-adjacent nodes that share common contextual attributes (e.g., the same Process ID, Component, or operational entity), irrespective of their temporal positions. This capability is crucial for accurately modeling the multivariate relationships in such a concurrent environment.

In summary, the empirical observation of frequent context switching underscores the limitation of prior studies and provides strong motivation for adopting a hypergraph model. Our hyperedge design is a direct response to this challenge, enabling LogHGAD to transcend temporal adjacency and robustly capture the complex, concurrent interactions inherent in modern AIoT systems.

4. Methodology

In this section, we propose a log anomaly detection method named Log Hypergraph Anomaly Detection (**LogHGAD**), which consists of four core modules: log preprocessing, log hypergraph generation, hypergraph convolutional network, and anomaly detection, as illustrated in Fig. 5. First, raw log data is collected from diverse sources (e.g., IoT devices, distributed databases, and service clusters). During preprocessing, the raw logs are first parsed and structured into log dataframe. Then, a set of log hypergraph snapshots based on log windows is obtained through three steps: log grouping, log representation, and hypergraph construction. The hypergraph is input into a hypergraph convolutional network to output the structural embeddings of hypergraph nodes. These structural embeddings are then fused with semantic embeddings to obtain final node representations, which undergo attention pooling to generate the overall hypergraph representation. In the anomaly detection stage, a downstream classifier determines whether there is an anomaly in the current log window based on the final representation of the given log hypergraph.

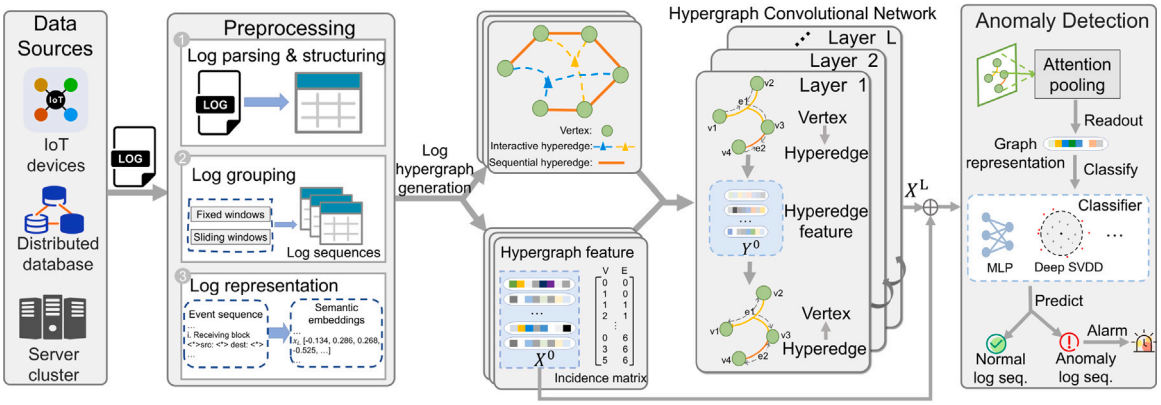
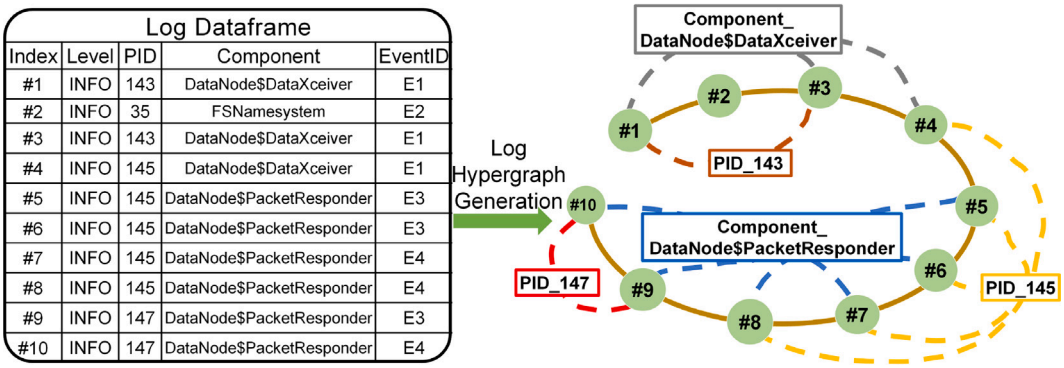


Fig. 5. The structure overview of LogHGAD.



Log Dataframe

Index	Level	PID	Component	EventID
#1	INFO	143	DataNode$DataXceiver	E1
#2	INFO	35	FSNamesystem	E2
#3	INFO	143	DataNode$DataXceiver	E1
#4	INFO	145	DataNode$DataXceiver	E1
#5	INFO	145	DataNode$PacketResponder	E3
#6	INFO	145	DataNode$PacketResponder	E3
#7	INFO	145	DataNode$PacketResponder	E4
#8	INFO	145	DataNode$PacketResponder	E4
#9	INFO	147	DataNode$PacketResponder	E3
#10	INFO	147	DataNode$PacketResponder	E4

Fig. 6. Process of generating log hypergraph from log dataframe. Green nodes represent hypergraph nodes. Solid lines denote sequential hyperedges generated according to chronological order, while dashed lines in different colors represent interactive hyperedges. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

4.1. Log grouping

Due to the concurrent execution characteristics of systems, the message sources or operation objects of log streams output by the system within a specific timeframe may differ. This leads to concurrent and interleaved log messages during output and collection. The primary goal of log grouping is to partition log sequences into distinct groups, ensuring complete and continuous event records for log messages with the same source or operation object.

We adopt session window grouping, where log dataframe is first divided into multiple session windows using identifiers. Each session window is uniquely identified by an identifier representing an execution path of the system. For example, in the HDFS dataset shown in Fig. 6, the block ID "blk_1608" is used for identification. This block ID, present in the log body of each message, denotes the file block ID operated by HDFS and is obtained through the log parsing steps described earlier.

Since the size of each session window varies, to facilitate subsequent hypergraph modeling and model training, after session window grouping, fixed window re-grouping is performed based on the chronological order of log messages within each window.

4.2. Log representation

We utilize the semantic embeddings of log events in each log message as feature representations, as shown in Fig. 7. Since log events are autonomously defined by software engineers during development to record system runtime states, leveraging their semantic information for anomaly detection is inherently reasonable. Log events are mostly composed of English words with distinct semantics, and may also include non-character tokens (e.g., numbers, punctuation, delimiters). Additionally, following software development conventions, some tokens are compound words (e.g., "SocketTimeoutException"). Drawing from prior work, we first preprocess log events by removing non-character tokens, then split compound words into individual words using Camel Case [45]. For example, "SocketTimeoutException" is split into "Socket", "Timeout", and "Exception". After preprocessing, we use FastText [46] word vectors pre-trained on the Common Crawl dataset [47] to extract semantic information for each token in log events.

Unlike prior studies [28–30] that adopted TF-IDF [48] aggregation, we employ average aggregation of word vectors for tokens in log events to obtain their semantic representations. This is because during model testing or in production environments, new log streams may contain unseen log events, where TF-IDF might assign excessively high weights to novel tokens, leading to biased detection. Simple average aggregation helps smooth noise and mitigate such biases.

4.3. Log hypergraph generation

The grouped log window sequences are used to construct a log hypergraph, where each log node is composed of the log event from a log message. A hypergraph node is uniquely determined by the tuple of the log message's timestamp and log event, with the semantic embedding of the log event serving as the node feature. The log hypergraph contains two types of hyperedges: Sequential Hyperedges and Interactive Hyperedges.

In Fig. 6, green nodes represent log nodes, and solid lines between nodes denote sequential hyperedges. Log nodes are sequentially connected to their temporal neighbors, ensuring the hypergraph remains

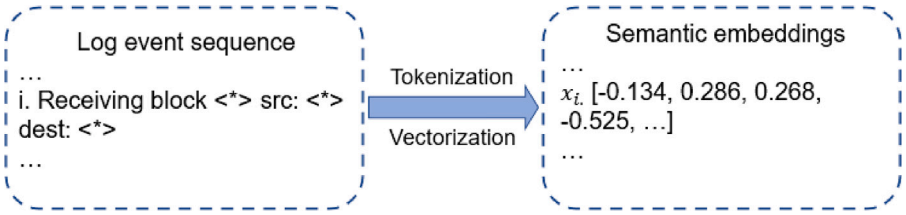


Fig. 7. The process of log representation.

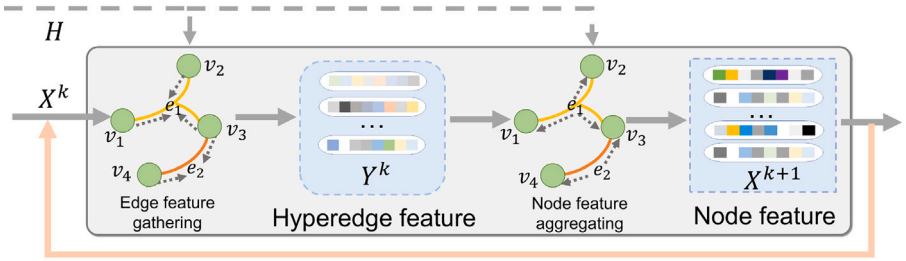


Fig. 8. The process of hypergraph convolution.

connected. Sequential hyperedges focus on the temporal correlation between logs. Interactive hyperedges capture implicit interaction relationships between log nodes. As shown in the left log dataframe of Fig. 6, log messages #1, #2, and #4 are all output by the "DataNode$DataXceiver" component. Logs #1 and #2 are executed by process PID 143, while log #4 is from PID 145. During operation, PID 145 interacts not only with "DataNode$DataXceiver" but also repeatedly with "DataNode$PacketResponder". Such interactions among processes, components, and process-component pairs cannot be described by traditional graph models. Thus, we introduce interactive hyperedges to connect any number of log nodes, enabling the representation of complex multivariate interactions within systems. As shown on the right side of Fig. 6, dashed lines represent interactive hyperedges, indicating information flow between log nodes via these connections.

Algorithm 1 is the construction process of the log hypergraph. The input is each log sequence divided by session identifiers and fixed windows, and the output is a log hypergraph. First, the node set, edge set, incidence matrix, and interaction relation set are initialized. In the first for loop of the algorithm (lines 3–14), log nodes are created, the interaction relations of each line of logs are extracted, and sequential hyperedges are established. The second for loop (lines 15–22) groups the log nodes according to the interaction relation set, and establishes common interaction hyperedges for the logs within the group. The time complexity of the entire algorithm is $O(nk)$, where n is the number of nodes in the window and k is the number of interaction relations.

At the same time, prior methods [28–31,37] often only focus on log events while ignoring log body parameters, causing the model to lose valuable information during the anomaly detection. This is because such parameters typically represent certain entities (e.g., IP addresses, file paths). We also establish hyperedge connections through log parameters among log nodes to model system-entity interactions. The specific steps are as follows: aggregate all log body parameters in the current log sequence window into a parameter set, remove pure numeric parameters from the set, and then group the log messages in the current log sequence window based on this set. Each group indicates that all log messages in the group are associated with a certain entity, for which hyperedge connections are established among the corresponding hypergraph nodes.

Algorithm 1 Log Hypergraph Construction.

Input: log window sequence L
Output: log hypergraph $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathcal{H})$

1: Initialize the node set $\mathcal{V}$, the hyperedge set $\mathcal{E}$ and the incidence matrix $\mathcal{H}$
2: Initialize the interaction relation set $\mathbb{R}$
3: **for** each log $L_i \in L$ **do**
4: Initialize a log node $v_i \leftarrow L_i$
5: Add v_i into $\mathcal{V}$
6: Extract all interactive relations r_s from L_i
7: Add r_s into $\mathbb{R}$
8: **if** $i \geq 1$ **then**
9: Initialize sequential hyperedge $se_i \leftarrow v_{i-1}v_i$
10: Add se_i into $\mathcal{E}$
11: $\mathcal{H} \leftarrow h(v_{i-1}, se_i) = 1$, $\mathcal{H} \leftarrow h(v_i, se_i) = 1$
12: i += 1
13: **end if**
14: **end for**
15: **for** each $r \in \mathbb{R}$ **do**
16: Extract interactive nodes $\mathcal{V}_r$ from $\mathcal{V}$ by r
17: Initialize interactive hyperedge $ie_r \leftarrow r$
18: Add ie_r into $\mathcal{E}$
19: **for** each node $v_r \in \mathcal{V}_r$ **do**
20: $H \leftarrow h(v_r, ie_r) = 1$
21: **end for**
22: **end for**
23: **return** $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathcal{H})$

4.4. Hypergraph convolutional network

Hypergraph convolutional network [43,44] effectively aggregates information from multi-node interactions by directly defining message passing (node ↔ hyperedge) on the hypergraph structure. During hypergraph convolution, the transition probabilities between connected nodes must be measured to enable feature propagation across the graph neural network. Hypergraph convolution is implemented based on two key assumptions: (1) features propagate more readily between nodes connected by a common hyperedge; (2) hyperedges with higher weights carry greater confidence during propagation. Consequently, the matrix representation of a single hypergraph convolution operation is defined as:

$$\mathcal{X}^{l+1} = \sigma(\mathcal{D}^{-1}\mathcal{H}\mathcal{W}\mathcal{B}^{-1}\mathcal{H}^T\mathcal{X}^l\theta) \tag{2}$$

where $\mathcal{X}^l$ and $\mathcal{X}^{l+1}$ denote the input feature matrices at layers l and $l+1$, respectively, and also serve as the node embedding matrices

output by the prior convolutional layer. θ is the weight matrix between layers l and $l+1$, and σ represents a non-linear activation function. To mitigate gradient explosion/vanishing from numerical instability when stacking multiple hypergraph convolutional layers, the formula applies row normalization using $\mathcal{D}^{-1}$ and $\mathcal{B}^{-1}$. The process of hypergraph convolution is shown in Fig. 8.

After passing through $\mathcal{L}$ hypergraph convolutional layers, enhanced hypergraph node embedding representations $\mathcal{X}^{\mathcal{L}}$ are obtained. These are then feature-fused with the original semantic embeddings $\mathcal{X}^0$ of the nodes to acquire the final node representations $\mathcal{X}$. Subsequently, through attention pooling, the final representation $\mathcal{Z}$ of the hypergraph is derived:

$$\mathcal{Z} = \mathcal{X}^T \cdot softmax(MLP(\mathcal{X})) \tag{3}$$

4.5. Anomaly detection

The hypergraph representation can be used with various downstream classification methods for log anomaly detection. To comprehensively evaluate the roles of log hypergraphs and hypergraph neural networks in LogHGAD, we implemented two types of downstream classifiers based on supervised learning and semi-supervised learning, respectively.

For the supervised learning classifier, we treat log anomaly detection as a binary classification task. Three Multi-Layer Perceptrons (MLPs) followed by a softmax function are used to output an anomaly score for each log hypergraph. If the anomaly score exceeds the predefined threshold (set to 0.5 in our experiments), the log sequence window represented by the current log hypergraph is flagged as anomalous, triggering an alert.

Additionally, since anomalies are rare during application operation, log data collections exhibit severe class imbalance between normal and anomalous log messages. For example, only 9.78% of hypergraphs in the Hadoop dataset [16,25] are anomalous (Table 2), far fewer than normal ones. We thus adopt Focal Loss [49] as the loss function, which dynamically weights samples to prioritize anomaly detection. The Focal Loss is defined as:

$$FL(p, y) = -\alpha(1-p)^{\gamma} log(p) - (1-\alpha)(1-y)p^{\gamma} log(1-p) \tag{4}$$

where y is the sample label, p is the model's predicted value (i.e., the anomaly score), α is the positive/negative sample weight balance factor, typically $\alpha \in [0, 1]$ and γ is the modulation factor controlling the weight difference between easy/hard samples, usually $\gamma \geq 0$. A larger γ more significantly attenuates the loss weight of easy-to-classify samples, making the model focus more on hard-to-classify samples.

For the semi-supervised learning classifier, we employ Deep SVDD [50], a deep learning-based one-class classification algorithm designed to detect anomalies in datasets. Its core principle involves mapping input data into a high-dimensional feature space via a neural network, then constructing a hypersphere of minimal volume in this space to enclose as many normal samples as possible while excluding anomalies. We use the loss function under Deep SVDD's soft-boundary paradigm as the optimization objective:

$$Loss = \frac{\tau}{\mathcal{N}} \sum_{n=0}^{\mathcal{N}} max(\|Z_n - c\|^2 - \mathcal{R}^2, 0) + \mathcal{R}^2 + \frac{\lambda}{2} \sum_{l=1}^{\mathcal{L}} \|\theta\|_F^2 \tag{5}$$

where c is the center of the hypersphere, $\|Z_n - c\|^2$ is the squared distance from the final representation of the log hypergraph to the hypersphere center, $\mathcal{R}$ is the trainable radius of the hypersphere, n is the number of log hypergraphs, and l is the number of layers in the hypergraph convolutional network. The first term in the formula represents the penalty for samples lying outside the hypersphere, where the hyperparameter τ balances the boundary, allowing some training data (both normal and anomalous samples) to be mapped outside the hypersphere. The second and third terms regularize the hypergraph convolutional network weights to prevent overfitting, with λ denoting the weight decay parameter.

During the model testing phase, the anomaly score of a log hypergraph is defined as the distance from its final representation to the hypersphere center:

$$Score(\mathcal{Z}) = \|\mathcal{Z} - c\| \tag{6}$$

If the anomaly score $Score(\mathcal{Z})$ is greater than the hypersphere radius $\mathcal{R}$, the current log hypergraph is identified as anomalous.

5. Experiments

In this section, we evaluate LogHGAD's[1] effectiveness and practicality. We introduce five public log datasets, seven baseline methods, and experimental setups, evaluating performance with multiple metrics. Ablation studies and efficiency analysis demonstrate LogHGAD's accuracy and deployment advantages.

5.1. Datasets

Following existing research, we utilize five widely adopted public log datasets [16]: BGL (Blue Gene/L supercomputer) [51], Hadoop [25], HDFS (Hadoop Distributed File System) [24], Spirit [52], and Thunderbird [51]. First, we extract log sequences for each session window using dataset-specific identifiers. For all datasets, we further partition these session windows into fixed windows of 50 log messages to construct log hypergraphs. Windows containing fewer than 50 logs are still processed into hypergraphs.

Table 2 summarizes the statistical information of the public log datasets used in this study. As shown in Table 2, these datasets exhibit diversity in terms of the number of log events and the anomaly hypergraph rate, which can better verify and evaluate the generalization ability of LogHGAD.

5.2. Baselines

To evaluate the effectiveness of LogHGAD, we compared it with seven state-of-the-art log anomaly detection methods on the aforementioned public log datasets as baselines. These methods include: quantitative statistics-based machine learning methods PCA [24], LogCluster [25], deep learning methods based on sequence models DeepLog [27], LogAnomaly [28], LogRobust [29], PLELog [30] and the state-of-the-art graph learning-based method Logs2Graphs [37], which converts logs into a weighted directed graph, leverages DiGCN model to extract graph structural features, and employs One-Class Deep SVDD for anomaly detection. Among them, PCA, LogCluster, DeepLog, LogAnomaly, and Logs2Graphs are unsupervised learning methods; PLELog is a semi-supervised learning method; and LogRobust is a supervised learning method.

5.3. Experimental setting

We implemented LogHGAD and its variants using Python 3.12.9 and PyTorch 2.6.0. The hypergraph neural network encoder of LogHGAD consists of two layers, followed by a classifier with 300 hidden units and a dropout rate of 0.5. The model was trained using the AdamW [53] optimizer with an initial learning rate of 0.001. We employed a ReduceLROnPlateau [54] learning rate scheduler, which halves the learning rate when the validation loss plateaus for three consecutive epochs. The batch size was set to 64, and the maximum number of training epochs was 30, with early stopping if the F1-Score on the validation set did not improve for five consecutive epochs. For each dataset, we use 70% of the samples for training, 10% for

[1] https://github.com/songchaocn/LogHGAD

Table 2
Log dataset statistics.

	BGL	Hadoop	HDFS	Spirit	Thunderbird
Total # of log messages	4,747,963	393,433	11,175,629	1,000,000	5,000,000
Total # of anomalies	348,460	17,249	359,929	289,905	226,095
Anomalous log messages %	7.34%	4.38%	3.22%	28.99%	4.52%
Total # of log events	762	517	47	839	1,009
Total # of hypergraphs	137,147	8,416	575,270	20,228	102,131
# of anomaly hypergraphs	54,183	823	136,740	8,309	13,605
Anomaly hypergraphs %	39.51%	9.78%	23.77%	41.08%	13.32%

validation, and 20% for testing. During training, we saved the model parameters that achieved the highest F1-Score on the validation set, as well as the parameters of the final model after all epochs. The reported results are based on the model (either the best or the final) that achieved the highest F1-Score on the test set.

5.4. Evaluation metrics

To evaluate the effectiveness of these methods, we follow prior studies and use accuracy, precision, recall, and F1-Score as metrics. In addition, based on the research in [29], we introduce specificity as an additional evaluation metric. This is because highly imbalanced log data with a small number of anomalies can hinder model performance. Specificity measures the probability of false positives of the model, that is, identifying normal log sequences as abnormal. High specificity means the model has a low false alarm rate. The specific calculation formulas for each metric are as follows:

- **Accuracy:** The proportion of log hypergraphs correctly predicted in all experiments to the total number of log hypergraphs detected as: $Accuracy = \frac{TP+TN}{TP+FP+TN+FN}$.
- **Precision:** The proportion of correctly identified anomalous log hypergraphs among all predicted anomalous log hypergraphs in all experiments as: $Precision = \frac{TP}{TP+FP}$.
- **Recall:** The proportion of correctly predicted anomalous log hypergraphs to the total actual anomalous log hypergraphs in all experiments as: $Recall = \frac{TP}{TP+FN}$.
- **Specificity:** The proportion of correctly predicted anomalous log hypergraphs to the total actual normal log hypergraphs in all experiments as: $Specificity = \frac{TN}{TN+FP}$.
- **F1-Score:** The F1-Score takes into account both the precision and recall of the model, defined as the harmonic mean of the model's precision and recall as: $F1\text{-}Score = \frac{2\times Precision\times Recall}{Precision+Recall}$.

5.5. Results and analysis

We first evaluate the effectiveness of LogHGAD and Baseline models on the above five public log datasets. The experimental results are shown in Table 3, where the top seven rows of each dataset table list the comparative Baseline methods sorted from oldest to newest, and the bottom two rows present our proposed LogHGAD methods. LogHGAD (MLP) denotes the supervised method with a 3-layer MLP as the downstream classifier, while LogHGAD (SVDD) represents the semi-supervised method using Deep SVDD as the downstream classifier. The highest results in each column are highlighted in bold.

Overall, the accuracy of the new methods generally outperforms that of the old methods across the five datasets, as the new methods fully address the shortcomings of the old ones and incorporate improvements. Supervised learning methods excel in accuracy compared to semi-supervised and unsupervised methods because they leverage labeled data effectively. For our proposed supervised classifier-based method, LogHGAD (MLP), its five evaluation metrics outperform the vast majority of methods on all five log datasets, and it achieves the best F1-Score on all five datasets. Although the supervised model LogRobust in the Baselines performs well on most datasets, its F1-Score on the Hadoop dataset is below 0.80 (0.781). As shown in Table 2, the Hadoop dataset has 1 to 2 orders of magnitude fewer logs than the other four datasets, and abnormal log data accounts for only 4.38%, meaning LogRobust may fail to fully mine the abnormal patterns in Hadoop logs.

For LogHGAD (SVDD) based on a semi-supervised classifier, it also achieves performance exceeding 0.90 on five evaluation metrics across the other four datasets except Hadoop. Compared with other semi-supervised or unsupervised methods, it outperforms these models on partial metrics, with scores approaching 1.0. Additionally, on the

Table 3
Performance comparison results of LogHGAD and baselines.

Dataset	Method	Metrics				
		Accuracy	Precision	Recall	Specificity	F1-Score
BGL	PCA	0.491	0.491	0.273	0.938	0.351
	LogCluster	0.928	0.939	0.639	0.991	0.761
	DeepLog	0.172	0.139	0.704	0.058	0.232
	LogAnomaly	0.234	0.137	0.631	0.148	0.226
	LogRobust	**0.999**	**0.999**	0.997	**0.999**	**0.998**
	PLELog	0.991	0.990	0.962	0.997	0.976
	Logs2Graphs	0.914	0.901	0.992	0.697	0.944
	LogHGAD (MLP)	0.998	0.998	**0.999**	**0.999**	**0.998**
	LogHGAD (SVDD)	0.994	0.986	**0.999**	0.991	0.993
Hadoop	PCA	0.871	0.133	**1.000**	0.868	0.235
	LogCluster	0.986	0.600	**1.000**	0.986	0.750
	DeepLog	0.623	0.050	**1.000**	0.616	0.095
	LogAnomaly	0.752	0.074	**1.000**	0.747	0.137
	LogRobust	0.877	0.904	0.687	0.966	0.781
	PLELog	0.927	0.214	**1.000**	0.925	0.352
	Logs2Graphs	0.405	0.940	0.390	0.636	0.551
	LogHGAD (MLP)	**0.991**	**0.949**	0.961	**0.994**	**0.955**
	LogHGAD (SVDD)	0.957	0.730	0.878	0.965	0.797
HDFS	PCA	0.860	0.860	0.817	0.997	0.838
	LogCluster	0.996	**1.000**	0.836	**1.000**	0.910
	DeepLog	0.968	0.404	**1.000**	0.967	0.576
	LogAnomaly	0.993	0.977	0.698	0.999	0.814
	LogRobust	**0.999**	0.999	0.996	**1.000**	0.997
	PLELog	0.994	0.998	0.726	0.999	0.840
	Logs2Graphs	0.738	0.250	0.867	0.724	0.388
	LogHGAD (MLP)	**0.999**	0.999	0.999	0.999	**0.999**
	LogHGAD (SVDD)	0.993	0.972	**1.000**	0.991	0.986
Spirit	PCA	0.859	0.689	0.965	0.813	0.804
	LogCluster	0.964	0.933	0.948	0.971	0.940
	DeepLog	0.392	0.392	0.999	0.336	0.563
	LogAnomaly	0.639	0.454	0.998	0.485	0.624
	LogRobust	0.902	0.993	0.679	**0.998**	0.807
	PLELog	0.950	0.868	0.984	0.936	0.922
	Logs2Graphs	0.949	0.934	**1.000**	0.810	0.966
	LogHGAD (MLP)	**0.997**	**0.996**	0.998	0.997	**0.997**
	LogHGAD (SVDD)	0.969	0.930	0.999	0.948	0.963
Thunderbird	PCA	0.504	0.004	0.002	0.708	0.003
	LogCluster	0.983	0.948	0.998	0.978	0.972
	DeepLog	0.517	0.374	**1.000**	0.321	0.545
	LogAnomaly	0.691	0.483	**1.000**	0.566	0.652
	LogRobust	0.998	0.997	0.998	0.998	0.997
	PLELog	0.964	0.891	0.998	0.950	0.942
	Logs2Graphs	0.823	0.673	**1.000**	0.721	0.804
	LogHGAD (MLP)	**0.999**	**1.000**	0.998	**0.999**	**0.999**
	LogHGAD (SVDD)	**0.999**	0.997	0.999	**0.999**	**0.999**

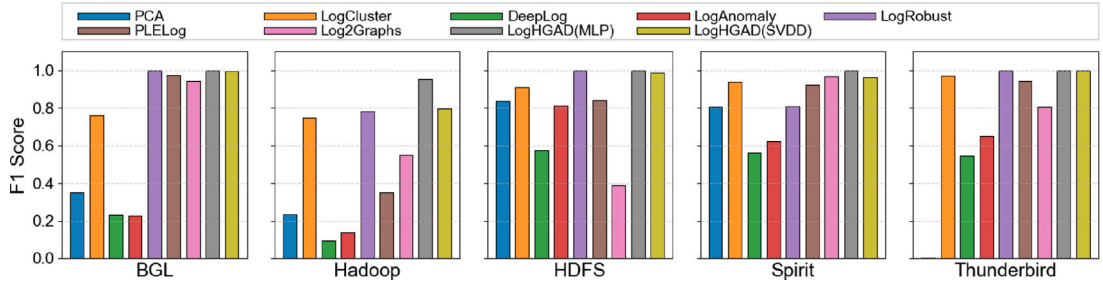


Fig. 9. F1-Score comparison between baseline methods and LogHGAD.

Hadoop, Spirit, and Thunderbird datasets, its F1-Score surpasses that of LogRobust. The above experimental results demonstrate that the proposed log hypergraph-based anomaly detection method can fully discover the majority of normal and abnormal operation patterns in log datasets.

Fig. 9 shows a bar chart comparing the F1-Scores of the 9 methods we compared on the above five public log datasets to more clearly demonstrate the superiority of our proposed LogHGAD. The left seven methods in each bar chart are the comparative Baseline methods, and the right two methods are our proposed LogHGAD (MLP) and LogHGAD (SVDD) methods. The highest value on each bar chart is highlighted in red. It can be seen that LogHGAD (MLP) achieves the best F1-Score on all five datasets, and LogHGAD (SVDD) has an F1-Score close to or equal to that of LogHGAD (MLP) on the other four datasets except Hadoop. On the Hadoop dataset, the F1-Score of LogHGAD (SVDD) also outperforms other Baseline methods.

Additionally, as pointed out in [23], highly imbalanced data with a small number of anomalies can hinder model performance. Commonly used metrics such as precision, recall, and F1-Score are insufficient for evaluating anomaly detection methods based on highly imbalanced log data. For example, the latest graph-based log anomaly detection method Logs2Graphs achieves a F1-Score of 0.944 on the BGL dataset, but its specificity is only 0.697. This indicates that although Logs2Graphs can accurately predict most anomalies with few omissions, it also has a large number of false positives, classifying nearly one-third of normal log sequences as anomalies, making the model almost inoperable in practical scenarios. In contrast, our proposed LogHGAD method, whether based on a supervised or semi-supervised classifier, has a specificity higher than 0.940, outperforming the vast majority of methods. Among them, LogHGAD (MLP) has better specificity than other methods on the four log datasets except HDFS. Additionally, except for the specificity of 0.994 on the Hadoop dataset, the specificity on other datasets reaches 0.999 (close to 1.0), indicating that LogHGAD (MLP) can identify almost all normal log sequences and meet the operational requirements in practical environments.

5.6. Ablation study

To further evaluate the impact of the log hypergraph in LogHGAD on the overall detection performance, we conducted ablation experiments on LogHGAD with a supervised classifier, specifically assessing the contribution of interactive hyperedges and the graph structure to the overall accuracy. The experimental results are shown in Fig. 10, which presents the precision, recall, and F1-Score of LogHGAD and its variants on the five public log datasets. Here, *LogHGAD w/o graph* refers to removing the log hypergraph from the LogHGAD model and instead using a Bi-GRU to input the log sequences. The structure of the *LogHGAD w/o graph* is similar to the aforementioned LSTM-based models, such as LogAnomaly. *LogHGAD w/o interactive hyperedges* indicates that only the interactive hyperedges were removed while retaining the sequential hyperedges to maintain graph connectivity. This setup aims to validate the core proposition of this paper.

From Fig. 10, we can observe that when LogHGAD does not use the hypergraph structure, the precision of the model decreases significantly only on the BGL dataset, while remaining relatively unchanged on the other datasets. However, the recall drops significantly across all five datasets. Notably, on the Hadoop dataset, the recall decreases by 0.227, indicating that after removing the hypergraph structure, the model increasingly misclassifies normal log sequences as anomalies, significantly reducing its ability to learn normal patterns. Additionally, by comparing the F1-Scores in Figs. 9 and 10, we can see that the accuracy of *LogHGAD w/o graph* outperforms some unsupervised methods, as it also uses labeled data for training. As for *LogHGAD w/o interactive hyperedges*, a noticeable decline in F1-Score is observed across all datasets. Except for Hadoop, the magnitude of this decrease is slightly smaller than that of *LogHGAD w/o graph*, suggesting that the incorporation of interactive hyperedges contributes to improved detection accuracy to some extent. The sharp decline in F1-Score on the Hadoop dataset implies that interactions among certain components or entities significantly influence the runtime system state. By incorporating interactive hyperedges, the model can more accurately perceive changes in the system state.

In conclusion, the log hypergraph and the hypergraph convolutional network play a crucial role in LogHGAD.

5.7. Hyperparameter study

In the model, fixed window size and the number of hypergraph convolutional layers are two key hyperparameters that affect performance. We tested the window size (ranging from 25 to 125 with a step size of 25) and the number of network layers (ranging from 1 to 5) on the Spirit dataset respectively, and visualized the results using line charts with five evaluation metrics to analyze the impact of hyperparameters. This dataset was selected because most of its session windows, after being divided by identifiers, contain more than 125 log messages, which meets the experimental requirements.

The experimental results in Fig. 11 show that regardless of whether the downstream classifier is MLP or Deep SVDD, all metrics of LogHGAD on Spirit are higher than 0.9 with a small variation range, indicating that the model has good robustness to different window sizes and can be stably applied to practical scenarios. The model achieves optimal performance with a smaller window size; however, as the window size increases, the scale of nodes and hyperedges in the log hypergraph expands, leading to a significant decrease in both accuracy and F1-Score for LogHGAD (MLP). Although LogHGAD (SVDD) shows significant fluctuations in precision and specificity, its accuracy and F1-Score remain largely stable.

Regarding the number of layers in Fig. 12, when the number of hypergraph convolutional layers increases from 1 to 2, LogHGAD (MLP) achieves the highest values in all four metrics except recall, demonstrating that the two-layer structure can better capture hypergraph features. A further increase in the number of layers results in a decrease in accuracy, recall, and F1-Score, indicating that an excessively deep network may cause over-smoothing, which weakens the ability to distinguish nodes and learn representations. In contrast, LogHGAD (SVDD) has strong adaptability to different numbers of layers, with generally stable metrics.

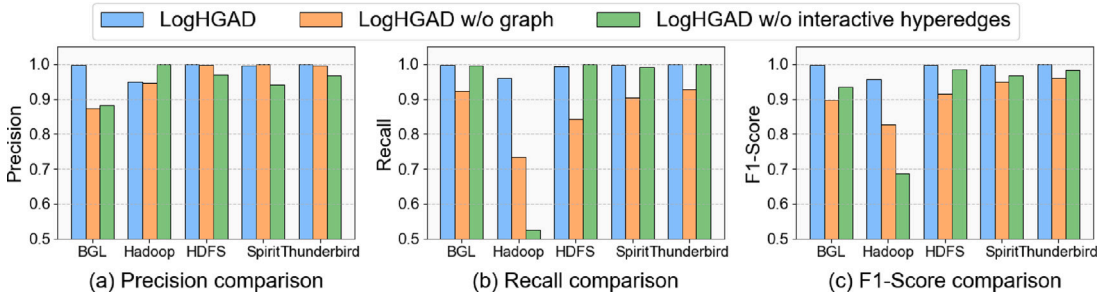


Fig. 10. Ablation study on graph structure.

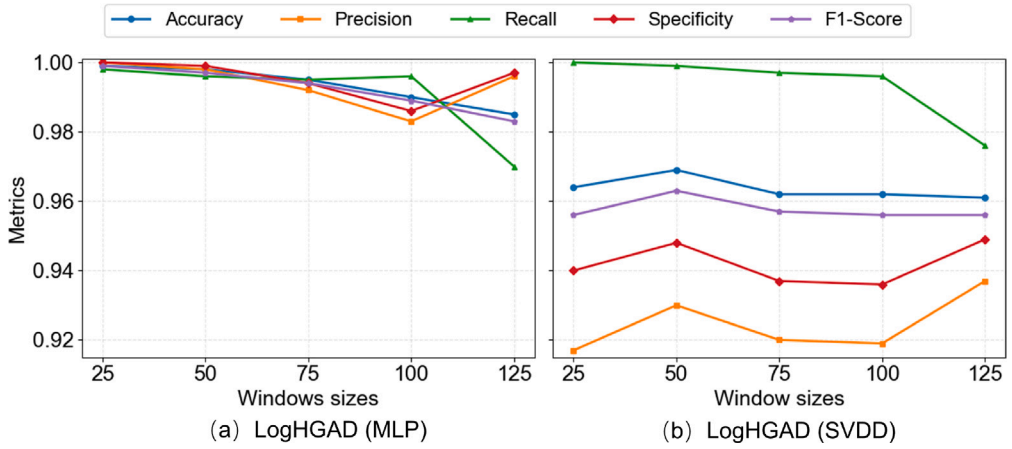


Fig. 11. Impact of different window sizes.

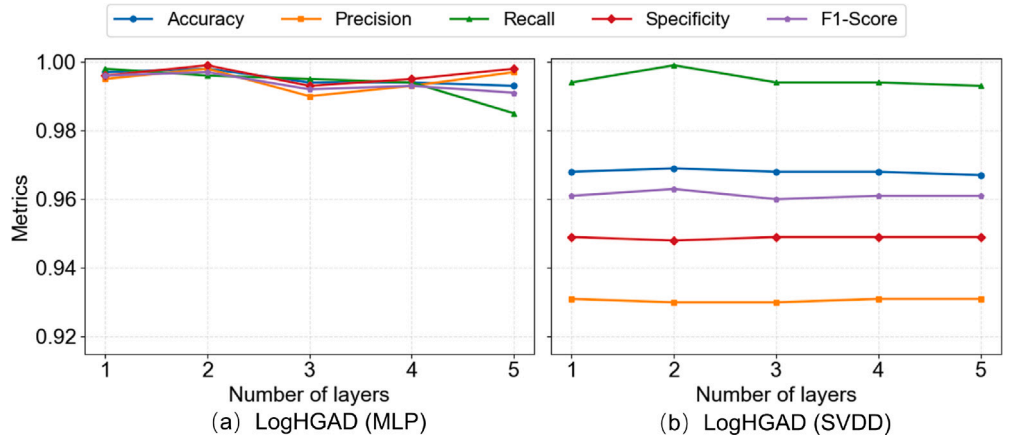


Fig. 12. Impact of different number of layers.

5.8. Efficiency analysis

To better demonstrate the superiority of our proposed LogHGAD, we also evaluated the efficiency of all methods on the HDFS dataset. Efficiency is another crucial criterion for the practical applicability of a method, as it determines whether a model can detect anomalies and trigger alerts in a timely manner in real-world environments. Table 4 presents the training and testing times of different methods on the HDFS dataset. The HDFS dataset contains 11,175,629 log messages, 70% of which were used for model training. All log messages were tested on the optimal models of each method, and the testing times were recorded. The testing process excluded preprocessing steps such as log data structuring, grouping, log representation, and model loading.

Table 4 shows that the two traditional machine learning-based methods (PCA and LogCluster) consume far less training and testing

Table 4
Efficiency comparison among different methods on the HDFS dataset.

Category	Method	Training time	Testing time
Machine Learning	PCA	0.12 s	0.15 s
	LogCluster	15 s	0.81 s
Deep Learning	DeepLog	39.77 m	20 s
	LogAnomaly	37.81 h	95 s
	LogRobust	21.63 h	300 s
	PLELog	1.49 h	275 s
Graph	Logs2Graphs	10.68 m	12 s
	LogHGAD (MLP)	25.53 m	13 s
	LogHGAD (SVDD)	16.62 m	9 s

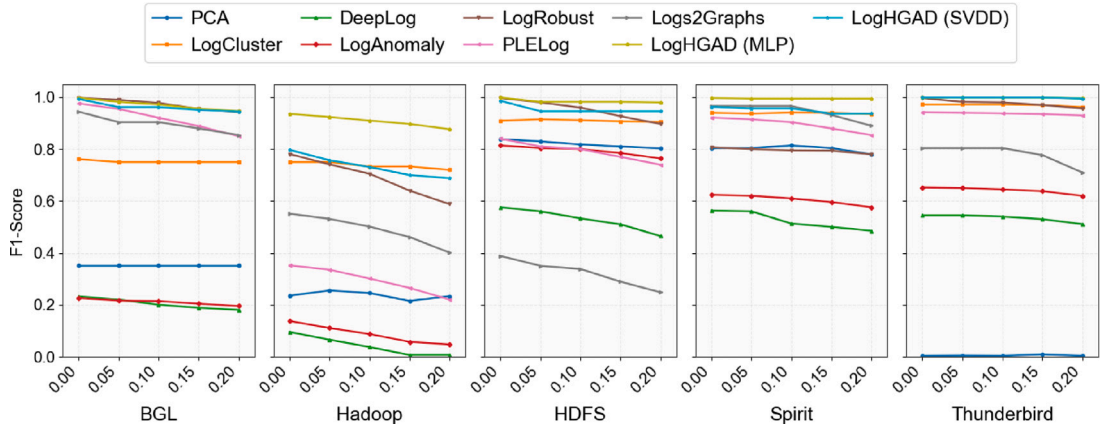


Fig. 13. Results with different ratios of noisy logs.

time than other methods. This is because these methods are based on lightweight machine learning models and only utilize statistical information about the number of log events, resulting in significantly lower computational complexity. However, as previously mentioned, the accuracy of these two methods depends on the dataset, and their test results exhibit instability. Additionally, the four LSTM-based methods have significantly longer training and testing times than other methods. This is due to their complex internal structures and temporal dependencies, which require more computational and storage resources to train a large number of parameters and maintain intermediate states. Although these methods can achieve good detection accuracy, their high training and testing overheads limit their practical applications.

In contrast, the graph-based Logs2Graphs and our two log hypergraph-based LogHGAD methods have shorter training and testing times than the LSTM-based methods, and these times are within an acceptable range. Although our two LogHGAD methods are slower in training time compared to Logs2Graphs, it can be seen from Fig. 9 that on the HDFS dataset, both of our methods achieve a performance improvement of approximately 0.6 over Logs2Graphs. Moreover, LogHGAD outperforms the other seven methods in all five evaluation metrics. In summary, our proposed LogHGAD is better suited for real-world scenarios than other methods.

5.9. Robustness analysis

To ensure the practicality and reliability of LogHGAD in real-world noisy environments, we systematically conducted robustness tests. We simulated practical deployment issues such as log loss, parsing errors, and data contamination by injecting different ratios of noise (5%, 10%, 15%, and 20%) into the test sets of the five datasets. To inject noise at a ratio r, we randomly selected $r\%$ of log messages from each test-set hypergraph and corrupted each one by applying, with equal probability, one of three strategies: random deletion, random replacement with a log from another session, or data duplication. We compared and analyzed the performance trends of LogHGAD against seven baseline methods. As shown in Fig. 13, the results fully demonstrate LogHGAD's superior resistance to interference compared to the seven baseline methods.

Across all datasets and noise levels, LogHGAD exhibited remarkable stability with minimal performance degradation. Under the most challenging condition of 20% noise contamination, LogHGAD (MLP) maintained an average F1-Score of 0.959 across the five datasets, representing only a 3.6% performance drop from the noise-free condition. This robustness was particularly evident in the HDFS dataset, where LogHGAD (MLP) preserved a near-perfect F1-Score of 0.98 even when one-fifth of the log data was randomly deleted or replaced, benefiting from its hypergraph structure's ability to infer missing information through preserved associative relationships.

Furthermore, comparative analysis revealed distinct robustness patterns among different methodological approaches. Statistical methods such as PCA showed minimal performance variation, but this came at the cost of consistently low baseline accuracy rather than representing true robustness. Sequence-based deep learning methods like DeepLog and LogAnomaly demonstrated significant sensitivity to noise, particularly those relying on long-term temporal dependencies showed the most substantial performance degradation. The graph-based method Logs2Graphs exhibited unstable performance fluctuations, with anomalous performance recovery phenomena in the BGL and Thunderbird datasets, possibly indicating overfitting to specific noise patterns. In all comparisons, both variants of LogHGAD consistently outperformed the baseline methods, with the supervised learning version (MLP) showing better stability in the data-scarce Hadoop dataset.

5.10. Anomaly explanation

This section explores the application of LogHGAD in log anomaly detection through a specific case study. We extract a log sequence from the HDFS dataset and construct a log hypergraph containing 28 nodes and 65 hyperedges. Among these, nodes with indices 15, 18, and 21 (indexing starts from 0) are abnormal, and these anomalies are all caused by communication issues between the system and a certain process in the network.

Fig. 14 illustrates the node anomaly scores computed using the LRP algorithm [55], visualized via the Python XGI toolkit [56]. In the figure, a hyperedge connecting exactly two nodes is represented by a solid line, while one linking more than two nodes is depicted as a convex hull. The anomaly level of each node is quantified by the color intensity. It can be clearly observed that the anomalous nodes — 15, 18, and 21 — exhibit darker colors, whereas the other nodes appear lighter. This result is consistent with the actual situation, indicating that LogHGAD not only successfully identifies the log nodes most relevant to anomalies but also effectively distinguishes between normal and abnormal nodes.

Moreover, nodes adjacent to anomalous ones may belong to the anomaly propagation chain, potentially being either affected by the anomaly or contributing to its cause. For instance, nodes 18, 22, 23, and 24 are connected by the same hyperedge, while nodes 21 and 22 are linked via a sequential hyperedge. Among these, nodes 18, 22, 23, and 24 show higher anomaly scores compared to other normal nodes, suggesting their possible relevance to the anomalies as secondary anomalous nodes. By examining the log contents corresponding to these nodes, the root cause of the anomalies and their impact scope can be further clarified.

In summary, LogHGAD not only facilitates rapid localization of anomalous nodes but also provides interpretable technical support for anomaly analysis.

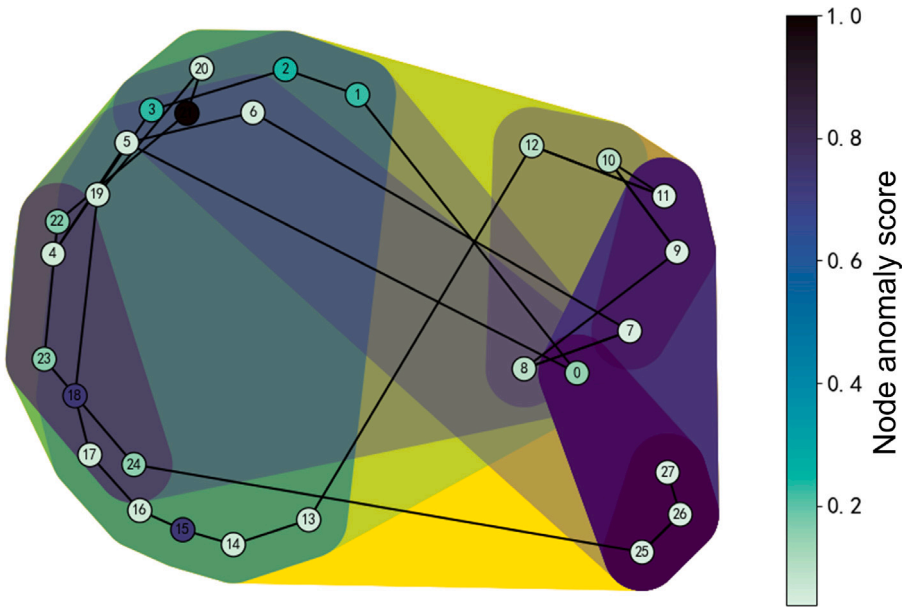


Fig. 14. Case study of anomaly explanation on a log hypergraph from HDFS dataset.

6. Conclusion

In this paper, we propose a hypergraph-based log anomaly detection method LogHGAD designed to enhance AIoT security by modeling the complex multivariate interaction relationships in log sessions through hypergraph structures. For the first time, it fuses log header information (such as processes and components) with log parameters, overcoming the defect of traditional methods in insufficient modeling of system context. Extensive experiments on five real-world log datasets demonstrate the superiority of LogHGAD. It outperforms existing methods in several key aspects. These include detection accuracy, with an F1-Score of up to 0.999, and high specificity, generally exceeding 0.99. Furthermore, it achieves high efficiency, with training and testing times significantly lower than those of sequence models. These results verify its strong adaptability to the dynamics and concurrency inherent in actual software systems.

Looking forward, our work opens up several promising avenues for future research. First, we plan to explore the application of LogHGAD beyond AIoT systems, such as in cloud computing infrastructures, blockchain networks, and large-scale microservice architectures, where complex component interactions are equally prevalent. Second, we intend to integrate multimodal data sources beyond logs, such as system performance metrics (e.g., CPU/memory usage) and network traffic flows, to construct a more comprehensive and robust anomaly detection framework. Furthermore, future work will also focus on the lightweight optimization of hypergraph structures and the development of online incremental learning mechanisms. These efforts will further adapt LogHGAD to evolving AIoT threat landscapes, ensuring real-time anomaly detection in distributed and scalable AIoT ecosystems. By bridging the gap between log analysis and AIoT security, LogHGAD will provide a robust foundation for proactive system monitoring and threat mitigation, ensuring continuous adaptability to dynamic security challenges in intelligent environments.

CRediT authorship contribution statement

Jiewei He: Writing – original draft, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Chao Song:** Writing – review & editing, Supervision, Project administration, Methodology. **Ruilin Hu:** Data curation, Conceptualization. **Zheng Ren:** Visualization, Validation, Resources, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Natural Science Foundation of China under Grant No. 62472070, 82241060, 62020106013; Sichuan Science and Technology Program under Grant No. 2024NSFSC0492; Sichuan Provincial Higher Education Talent Development Quality and Teaching Reform Project, China under Grant No. JG2024-0001 and JG2024-0939; the National Key R&D Program of China under Grant 2021YFB3101303 and 2021YFB3101302.

References

[1] D. Muhammed, E. Ahvar, S. Ahvar, M. Trocan, M. Montpetit, R. Ehsani, Artificial intelligence of things (AIoT) for smart agriculture: A review of architectures, technologies and solutions, J. Netw. Comput. Appl. 228 (2024) 103905, http://dx.doi.org/10.1016/J.JNCA.2024.103905.

[2] S. Zhang, Y. Pan, Q. Liu, Z. Yan, K.R. Choo, G. Wang, Backdoor attacks and defenses targeting multi-domain AI models: A comprehensive review, ACM Comput. Surv. 57 (4) (2025) 87:1–87:35, http://dx.doi.org/10.1145/3704725.

[3] F. Dong, X. Ge, Q. Li, J. Zhang, D. Shen, S. Liu, X. Liu, G. Li, F. Wu, J. Luo, PADP-FedMeta: A personalized and adaptive differentially private federated meta learning mechanism for AIoT, J. Syst. Archit. 134 (2023) 102754, http://dx.doi.org/10.1016/J.SYSARC.2022.102754.

[4] K. Li, Integrated analysis of reliability, power, and performance for IoT devices and servers, J. Syst. Archit. 154 (2024) 103216, http://dx.doi.org/10.1016/J.SYSARC.2024.103216.

[5] Y. Gao, L. Chen, J. Han, G. Wu, S. Liu, Similarity-based deduplication and secure auditing in IoT decentralized storage, J. Syst. Archit. 142 (2023) 102961, http://dx.doi.org/10.1016/J.SYSARC.2023.102961.

[6] H.N. Noura, O. Salman, A. Chehab, R. Couturier, DistLog: A distributed logging scheme for IoT forensics, Ad Hoc Networks 98 (2020) http://dx.doi.org/10.1016/J.ADHOC.2019.102061.

[7] S. Wang, Q. Liu, Y. Xu, H. Jiang, J. Wu, T. Wang, T. Peng, G. Wang, Protecting inference privacy with accuracy improvement in mobile-cloud deep learning, IEEE Trans. Mob. Comput. 23 (6) (2024) 6522–6537, http://dx.doi.org/10.1109/TMC.2023.3323450.

[8] H. Nguyen, R. Ivanov, L.T.X. Phan, O. Sokolsky, J. Weimer, I. Lee, LogSafe: Secure and scalable data logger for IoT devices, in: Proceedings of IEEE/ACM 3th International Conference on Internet-of-Things Design and Implementation, IoTDI, 2018, pp. 141–152, http://dx.doi.org/10.1109/IOTDI.2018.00023.

[9] Z. Zhang, C. Johnson, N. Venkatasubramanian, S. Ren, Process scenario discovery from event logs based on activity and timing information, J. Syst. Archit. 125 (2022) 102435, http://dx.doi.org/10.1016/J.SYSARC.2022.102435.

[10] R. Patil, S. Muneeswaran, V. Sachidananda, M. Gurusamy, E-Audit: Distinguishing and investigating suspicious events for APTs attack detection, J. Syst. Archit. 144 (2023) 102988, http://dx.doi.org/10.1016/J.SYSARC.2023.102988.

[11] B. Li, S. Ma, R. Deng, K.R. Choo, J. Yang, Federated anomaly detection on system logs for the internet of things: A customizable and communication-efficient approach, IEEE Trans. Netw. Serv. Manag. 19 (2) (2022) 1705–1716, http://dx.doi.org/10.1109/TNSM.2022.3152620.

[12] C. Song, Z. Ren, R. Hu, L. Lu, Enhancing IoT compliance checking with distributed process mining: A scalable framework for log data streams, in: Proceedings of the 18th China Conference on Wireless Sensor Networks, CWSN, 2024, pp. 130–140.

[13] Q. Liu, Y. Peng, Q. Xu, H. Jiang, J. Wu, T. Wang, T. Peng, G. Wang, S. Zhang, MARS: Enabling verifiable range-aggregate queries in multi-source environments, IEEE Trans. Dependable Secur. Comput. 21 (4) (2024) 1994–2011, http://dx.doi.org/10.1109/TDSC.2023.3299337.

[14] S. He, P. He, Z. Chen, T. Yang, Y. Su, M.R. Lyu, A survey on automated log analysis for reliability engineering, ACM Comput. Surv. 54 (6) (2022) 130:1–130:37, http://dx.doi.org/10.1145/3460345.

[15] S. He, X. Zhang, P. He, Y. Xu, L. Li, Y. Kang, M. Ma, Y. Wei, Y. Dang, S. Rajmohan, Q. Lin, An empirical study of log analysis at microsoft, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE, 2022, pp. 1465–1476, http://dx.doi.org/10.1145/3540250.3558963.

[16] J. Zhu, S. He, P. He, J. Liu, M.R. Lyu, Loghub: A large collection of system log datasets for AI-driven log analytics, in: Proceedings of the 34th IEEE International Symposium on Software Reliability Engineering, ISSRE, 2023, pp. 355–366, http://dx.doi.org/10.1109/ISSRE59848.2023.00071.

[17] H. Mi, H. Wang, Y. Zhou, M.R. Lyu, H. Cai, Toward fine-grained, unsupervised, scalable performance diagnosis for production cloud computing systems, IEEE Trans. Parallel Distrib. Syst. 24 (6) (2013) 1245–1255, http://dx.doi.org/10.1109/TPDS.2013.21.

[18] J. Xu, R. Yang, Y. Huo, C. Zhang, P. He, DivLog: Log parsing with prompt enhanced in-context learning, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE, 2024, pp. 199:1–199:12, http://dx.doi.org/10.1145/3597503.3639155.

[19] S. Yu, P. He, N. Chen, Y. Wu, Brain: Log parsing with bidirectional parallel tree, IEEE Trans. Serv. Comput. 16 (5) (2023) 3224–3237, http://dx.doi.org/10.1109/TSC.2023.3270566.

[20] P. He, J. Zhu, Z. Zheng, M.R. Lyu, Drain: An online log parsing approach with fixed depth tree, in: Proceedings of 2017 IEEE International Conference on Web Services, ICWS, 2017, pp. 33–40, http://dx.doi.org/10.1109/ICWS.2017.13.

[21] R. Christensen, F. Li, Adaptive log compression for massive log data, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD, 2013, pp. 1283–1284, http://dx.doi.org/10.1145/2463676.2465341.

[22] B. Yu, J. Yao, Q. Fu, Z. Zhong, H. Xie, Y. Wu, Y. Ma, P. He, Deep learning or classical machine learning? An empirical study on log-based anomaly detection, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE, 2024, pp. 35:1–35:13, http://dx.doi.org/10.1145/3597503.3623308.

[23] V. Le, H. Zhang, Log-based anomaly detection with deep learning: How far are we? in: Proceedings of the 44th IEEE/ACM International Conference on Software Engineering, ICSE, 2022, pp. 1356–1367, http://dx.doi.org/10.1145/3510003.3510155.

[24] W. Xu, L. Huang, A. Fox, D.A. Patterson, M.I. Jordan, Detecting large-scale system problems by mining console logs, in: Proceedings of the 22nd ACM Symposium on Operating Systems Principles, SOSP, 2009, pp. 117–132, http://dx.doi.org/10.1145/1629575.1629587.

[25] Q. Lin, H. Zhang, J. Lou, Y. Zhang, X. Chen, Log clustering based problem identification for online service systems, in: Proceedings of the 38th International Conference on Software Engineering, ICSE, 2016, pp. 102–111, http://dx.doi.org/10.1145/2889160.2889232.

[26] J. Lou, Q. Fu, S. Yang, Y. Xu, J. Li, Mining invariants from console logs for system problem detection, in: Proceedings of the 2010 USENIX Annual Technical Conference, ATC, 2010.

[27] M. Du, F. Li, G. Zheng, V. Srikumar, DeepLog: Anomaly detection and diagnosis from system logs through deep learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS, 2017, pp. 1285–1298, http://dx.doi.org/10.1145/3133956.3134015.

[28] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, R. Zhou, LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs, in: Proceedings of the 28th International Joint Conference on Artificial Intelligence, IJCAI, 2019, pp. 4739–4745, http://dx.doi.org/10.24963/IJCAI.2019/658.

[29] X. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Z. Li, J. Chen, X. He, R. Yao, J. Lou, M. Chintalapati, F. Shen, D. Zhang, Robust log-based anomaly detection on unstable log data, in: Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE, 2019, pp. 807–817, http://dx.doi.org/10.1145/3338906.3338931.

[30] L. Yang, J. Chen, Z. Wang, W. Wang, J. Jiang, X. Dong, W. Zhang, Semi-supervised log-based anomaly detection via probabilistic label estimation, in: Proceedings of 43rd IEEE/ACM International Conference on Software Engineering, ICSE, 2021, pp. 1448–1460, http://dx.doi.org/10.1109/ICSE43902.2021.00130.

[31] X. Wang, J. Song, X. Zhang, J. Tang, W. Gao, Q. Lin, LogOnline: A semi-supervised log-based anomaly detector aided with online learning mechanism, in: Proceedings of 38th IEEE/ACM International Conference on Automated Software Engineering, ASE, 2023, pp. 141–152, http://dx.doi.org/10.1109/ASE56229.2023.00043.

[32] M. Landauer, S. Onder, F. Skopik, M. Wurzenberger, Deep learning for anomaly detection in log data: A survey, 2022, http://dx.doi.org/10.48550/ARXIV.2207.03820, CoRR abs/2207.03820. arXiv:2207.03820.

[33] V. Le, H. Zhang, Log-based anomaly detection without log parsing, in: Proceedings of 36th IEEE/ACM International Conference on Automated Software Engineering, ASE, 2021, pp. 492–504, http://dx.doi.org/10.1109/ASE51524.2021.9678773.

[34] S. Lu, X. Wei, Y. Li, L. Wang, Detecting anomaly in big data system logs using convolutional neural network, in: Proceedings of 2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress, DASC/PiCom/DataCom/CyberSciTech, 2018, pp. 151–158, http://dx.doi.org/10.1109/DASC/PICOM/DATACOM/CYBERSCITEC.2018.00037.

[35] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering, ICSE, 2022, pp. 623–634, http://dx.doi.org/10.1145/3510003.3510180.

[36] Y. Xie, H. Zhang, M.A. Babar, LogGD: Detecting anomalies from system logs with graph neural networks, in: Proceedings of the 22nd IEEE International Conference on Software Quality, Reliability and Security, QRS, 2022, pp. 299–310, http://dx.doi.org/10.1109/QRS57517.2022.00039.

[37] Z. Li, J. Shi, M. van Leeuwen, Graph neural networks based log anomaly detection and explanation, in: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, ICSE, 2024, pp. 306–307, http://dx.doi.org/10.1145/3639478.3643084.

[38] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, M.R. Lyu, Tools and benchmarks for automated log parsing, in: Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice, ICSE, 2019, pp. 121–130, http://dx.doi.org/10.1109/ICSE-SEIP.2019.00021.

[39] Z. Jiang, J. Liu, J. Huang, Y. Li, Y. Huo, J. Gu, Z. Chen, J. Zhu, M.R. Lyu, A large-scale evaluation for log parsing techniques: How far are we? in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA, 2024, pp. 223–234, http://dx.doi.org/10.1145/3650212.3652123.

[40] Z.A. Khan, D. Shin, D. Bianculli, L.C. Briand, Guidelines for assessing the accuracy of log message template identification techniques, in: Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering, ICSE, 2022, pp. 1095–1106, http://dx.doi.org/10.1145/3510003.3510101.

[41] S. Agarwal, K. Branson, S.J. Belongie, Higher order learning with graphs, in: Proceedings of Machine Learning, Proceedings of the Twenty-Third International Conference, ICML, in: ACM International Conference Proceeding Series, vol. 148, 2006, pp. 17–24, http://dx.doi.org/10.1145/1143844.1143847.

[42] C. Zhang, S. Hu, Z.G. Tang, T.H. Chan, Re-revisiting learning on hypergraphs: Confidence interval and subgradient method, in: Proceedings of the 34th International Conference on Machine Learning, Vol. 70, ICML, 2017, pp. 4026–4034.

[43] S. Bai, F. Zhang, P.H.S. Torr, Hypergraph convolution and hypergraph attention, Pattern Recognit. 110 (2021) 107637, http://dx.doi.org/10.1016/J.PATCOG.2020.107637.

[44] Y. Feng, H. You, Z. Zhang, R. Ji, Y. Gao, Hypergraph neural networks, in: Proceedings of the 33rd AAAI Conference on Artificial Intelligence, AAAI, 2019, pp. 3558–3565, http://dx.doi.org/10.1609/AAAI.V33I01.33013558.

[45] B. Dit, L. Guerrouj, D. Poshyvanyk, G. Antoniol, Can better identifier splitting techniques help feature location? in: Proceedings of the 19th IEEE International Conference on Program Comprehension, ICPC, 2011, pp. 11–20, http://dx.doi.org/10.1109/ICPC.2011.47.

[46] A. Joulin, E. Grave, P. Bojanowski, M. Douze, H. Jégou, T. Mikolov, FastText.zip: Compressing text classification models, 2016, CoRR abs/1612.03651. arXiv:1612.03651.

[47] T. Mikolov, E. Grave, P. Bojanowski, C. Puhrsch, A. Joulin, Advances in pre-training distributed word representations, in: Proceedings of the Eleventh International Conference on Language Resources and Evaluation, LREC, 2018.

[48] G. Salton, C. Buckley, Term-weighting approaches in automatic text retrieval, Inf. Process. Manag. 24 (5) (1988) 513–523, http://dx.doi.org/10.1016/0306-4573(88)90021-0.

[49] T. Lin, P. Goyal, R.B. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: Proceedings of IEEE International Conference on Computer Vision, ICCV, 2017, pp. 2999–3007, http://dx.doi.org/10.1109/ICCV.2017.324.

[50] L. Ruff, N. Görnitz, L. Deecke, S.A. Siddiqui, R.A. Vandermeulen, A. Binder, E. Müller, M. Kloft, Deep one-class classification, in: Proceedings of the 35th International Conference on Machine Learning, Vol. 80, ICML, 2018, pp. 4390–4399.

[51] A.J. Oliner, J. Stearley, What supercomputers say: A study of five system logs, in: Proceedings of the 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN, 2007, pp. 575–584, http://dx.doi.org/10.1109/DSN.2007.103.

[52] J. Stearley, A.J. Oliner, Bad words: Finding faults in spirit's syslogs, in: Proceedings of the 8th IEEE International Symposium on Cluster Computing and the Grid, CCGrid, 2008, pp. 765–770, http://dx.doi.org/10.1109/CCGRID.2008.107.

[53] PyTorch Contributors, torch.optim.AdamW, 2025, https://docs.pytorch.org/docs/2.6/generated/torch.optim.AdamW.html. (Accessed 30 June 2025).

[54] PyTorch Contributors, torch.optim.lr_scheduler.ReduceLROnPlateau, 2025, https://docs.pytorch.org/docs/2.6/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html. (Accessed 30 June 2025).

[55] S. Bach, A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, W. Samek, On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation, PLoS ONE 10 (7) (2015) e0130140.

[56] N.W. Landry, M. Lucas, I. Iacopini, G. Petri, A. Schwarze, A. Patania, L. Torres, XGI: A python package for higher-order interaction networks, J. Open Source Softw. 8 (85) (2023) 5162, http://dx.doi.org/10.21105/joss.05162.



Jiewei He received the B.S. degree in Computer Science from Hangzhou City University (HZCU), China, in 2021. His current research interests include graph learning and anomaly detection.



Chao Song received his Ph.D. degree in Computer Science from the University of Electronic Science and Technology of China (UESTC), Chengdu, in 2009. He is currently an associate professor in the School of Computer Science and Engineering at UESTC. His main research interests include mobile computing, vehicular networks, and distributed computing.



Ruilin Hu received the M.S. degree from Xihua University, China, in 2022. He is currently working toward the Ph.D. degree with School of Computer Science and Engineering, University of Electronic Science and Technology of China (UESTC). His current research interests include graph computing and graph stream mining.



Zheng Ren received the B.S. degree from University of Electronic Science and Technology of China (UESTC), China, in 2022. His current research interests include graph representation learning and recommendation systems.